

Requirements Document

Introduction

This feature involves creating a Spring Boot backend service for an online school management system designed to teach bootcamp students. The system manages five core entities: Student, Teacher, Class, Course, and Registration. Both Students and Teachers inherit from a base User entity. The system provides comprehensive CRUD operations for all entities and manages relationships between them, including student course registrations, class-teacher assignments, and class-course associations.

Requirements

Requirement 1

User Story: As a bootcamp instructor, I want a complete Spring Boot backend with entity management, so that students can learn real-world API development patterns.

Acceptance Criteria

1. WHEN the application starts THEN the system SHALL initialize a Spring Boot application with proper configuration
2. WHEN the application is running THEN the system SHALL expose RESTful APIs on standard HTTP ports
3. WHEN entities are created THEN the system SHALL use JPA annotations for database mapping
4. WHEN the application connects to database THEN the system SHALL use Spring Data JPA for data persistence
5. WHEN the application starts THEN the system SHALL use H2 in-memory database for data storage
6. WHEN in development mode THEN the system SHALL enable H2 console for database inspection

Requirement 2

User Story: As a developer, I want properly structured entities with inheritance, so that the data model reflects real-world relationships.

Acceptance Criteria

1. WHEN defining entities THEN the system SHALL create a base User entity with common attributes (id, name, email, createdAt, updatedAt)
2. WHEN creating Student entity THEN the system SHALL extend User and include student-specific attributes (studentId, enrollmentDate)
3. WHEN creating Teacher entity THEN the system SHALL extend User and include teacher-specific attributes (employeeId, department, hireDate)
4. WHEN creating Course entity THEN the system SHALL include attributes (id, name, description, credits, duration)
5. WHEN creating Class entity THEN the system SHALL include attributes (id, name, semester, year, maxCapacity)

6. WHEN creating Registration entity THEN the system SHALL include attributes (id, registrationDate, status, grade)

Requirement 3

User Story: As a system administrator, I want to manage entity relationships, so that the school data maintains referential integrity.

Acceptance Criteria

1. WHEN a Student registers for a Course THEN the system SHALL create a Registration entity linking them
2. WHEN a Class is created THEN the system SHALL allow assignment of exactly one Teacher
3. WHEN a Class is created THEN the system SHALL allow multiple Students to be enrolled
4. WHEN a Class is created THEN the system SHALL allow multiple Courses to be associated
5. WHEN entities are deleted THEN the system SHALL handle cascading operations appropriately
6. WHEN relationships are queried THEN the system SHALL support bidirectional navigation

Requirement 4

User Story: As a client application developer, I want comprehensive CRUD APIs for all entities, so that I can build frontend applications.

Acceptance Criteria

1. WHEN accessing User endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
2. WHEN accessing Student endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
3. WHEN accessing Teacher endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
4. WHEN accessing Course endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
5. WHEN accessing Class endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
6. WHEN accessing Registration endpoints THEN the system SHALL provide GET, POST, PUT, DELETE operations
7. WHEN API calls are made THEN the system SHALL return appropriate HTTP status codes
8. WHEN API calls fail THEN the system SHALL return meaningful error messages

Requirement 5

User Story: As a developer, I want proper API documentation and validation, so that the APIs are easy to use and maintain.

Acceptance Criteria

1. WHEN API endpoints are defined THEN the system SHALL include input validation annotations
2. WHEN invalid data is submitted THEN the system SHALL return validation error messages

3. WHEN API responses are returned THEN the system SHALL use consistent JSON structure
4. WHEN exceptions occur THEN the system SHALL handle them gracefully with proper error responses
5. WHEN the application runs THEN the system SHALL provide API documentation (Swagger/OpenAPI)

Requirement 6

User Story: As a bootcamp student, I want to see advanced query capabilities, so that I can learn complex database operations.

Acceptance Criteria

1. WHEN querying Students THEN the system SHALL support finding students by class
2. WHEN querying Teachers THEN the system SHALL support finding teachers by department
3. WHEN querying Courses THEN the system SHALL support finding courses by credits or duration
4. WHEN querying Classes THEN the system SHALL support finding classes by semester and year
5. WHEN querying Registrations THEN the system SHALL support finding registrations by status
6. WHEN complex queries are needed THEN the system SHALL demonstrate custom repository methods